

Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement **Backward Chaining** and determine whether a given query can be proved from the available facts and rules.

Algorithm

1. Start.
2. Define the facts and rules in the knowledge base.
3. Give a query or goal to be proved.
4. Check whether the goal is a known fact.
5. If not, find a rule whose conclusion matches the goal.
6. Check whether all conditions of that rule can be proved.
7. Continue recursively until the required facts are found.
8. If all conditions are satisfied, return **true**; otherwise return **false**.
9. Stop.

Prolog Program

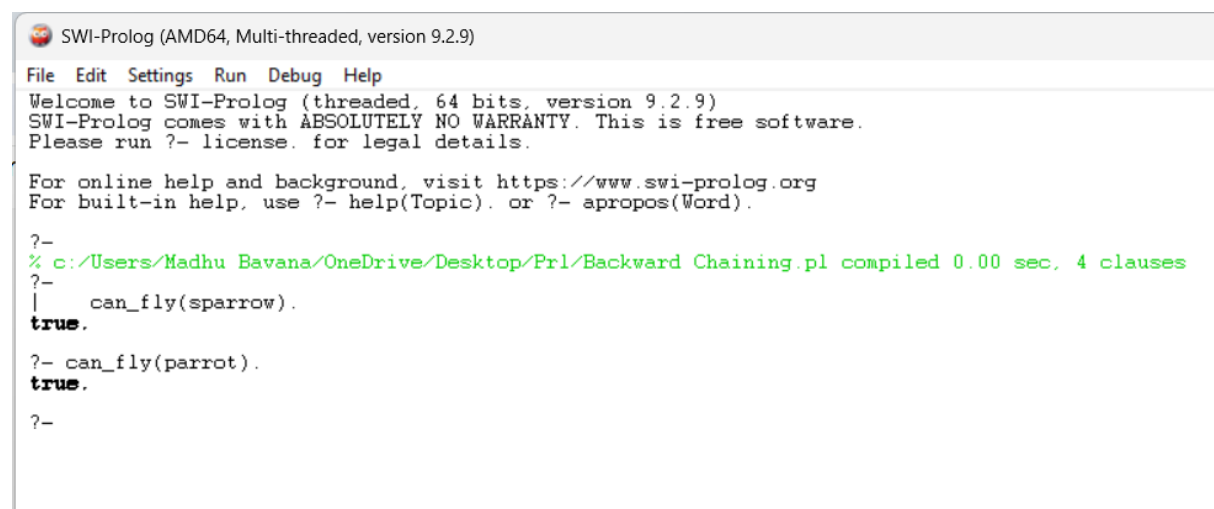
```
% Backward Chaining

bird(sparrow).
bird(parrot).

has_wings(X) :-
    bird(X).

can_fly(X) :-
    has_wings(X).
```

OUTPUT



```
SWI-Prolog (AMD64, Multi-threaded, version 9.2.9)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

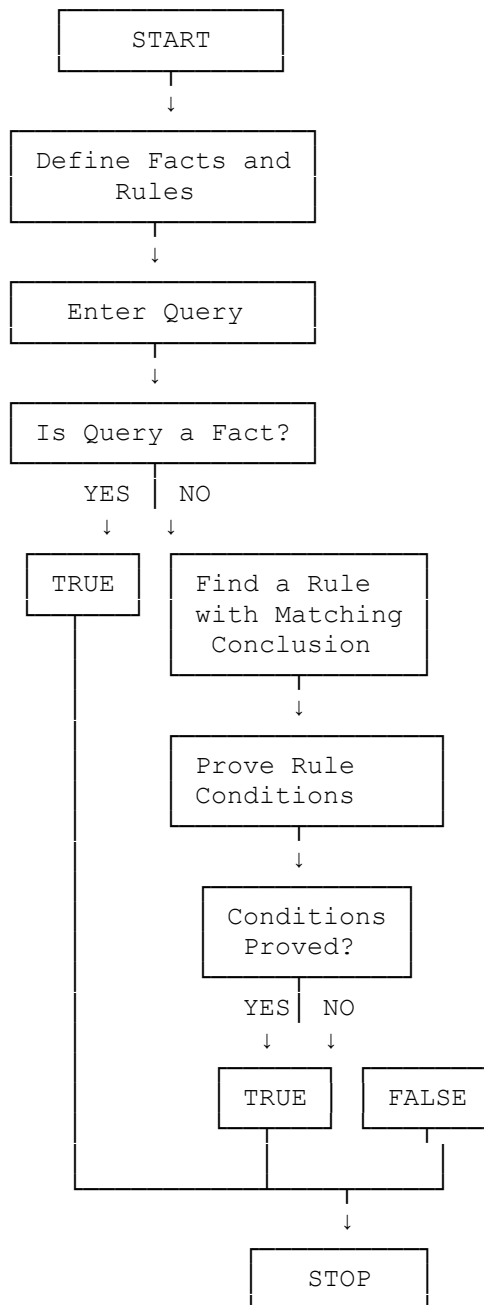
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% c:/Users/Madhu Bavana/OneDrive/Desktop/Prl/Backward Chaining.pl compiled 0.00 sec, 4 clauses
?-
|   can_fly(sparrow).
true.

?- can_fly(parrot).
true.

?-
```

Flowchart



Conclusion

Thus, the **Backward Chaining algorithm** is successfully implemented in Prolog by starting with the goal and working backward through the rules until the required facts are proved.